

Análise Arquitetural do Sistema de Xadrez Orientado a Objetos

Documentação Teórica de Decisões de Projeto e Princípios de POO

1. Introdução

O desenvolvimento de um motor de jogo de xadrez funcional na linguagem Java representa um excelente desafio prático para a aplicação rigurosa dos pilares da Programação Orientada a Objetos (POO). Mais do que implementar as regras de um jogo de tabuleiro, o objetivo primordial deste projeto reside no estabelecimento de uma arquitetura limpa, extensível, de fácil manutenção e conceitualmente sólida.

Este documento detalha as justificativas técnicas por trás da estrutura de classes adotada, o uso estratégico dos conceitos nativos da POO, a resolução algorítmica de mecânicas avançadas temporais/espaciais e as escolhas de design arquitetural aplicadas de forma autônoma para elevar a robustez do software.

2. Os Pilares de POO Aplicados ao Projeto

A flexibilidade e a resiliência do sistema baseiam-se na aplicação combinada dos quatro pilares fundamentais da Orientação a Objetos. A falha no emprego correto de qualquer um deles resultaria em um código altamente acoplado e de difícil expansão.

2.1. Abstração

A abstração permitiu isolar a complexidade genérica de um jogo de tabuleiro da mecânica estrita do xadrez. Isso se materializa na divisão estrutural entre a classe genérica `Piece` (no pacote base do tabuleiro) e a classe especializada `ChessPiece`.

A definição do método abstrato `public abstract boolean[][] possibleMoves();` na classe mãe força cada peça subsequente a assinar um contrato comportamental, abstraindo a lógica de varredura matricial para quem consome o método de movimentação no loop principal do jogo.

2.2. Herança

Para evitar a duplicação maciça de código, o mecanismo de herança foi empregado criando uma árvore hierárquica bem definida. Todas as peças específicas (`Pawn`, `King`, `Rook`, `Bishop`, `Knight` e `Queen`) herdam diretamente de `ChessPiece`.

Com isso, atributos essenciais como `color`, `board`, `position` e o recém-introduzido `moveCount` são centralizados na classe superior, garantindo reaproveitamento estrutural e permitindo que modificações transversais reflitam de forma imediata em todo o conjunto de entidades.

2.3. Encapsulamento

O encapsulamento protege o estado interno do jogo contra manipulações externas indevidas ou incoerentes. No projeto, os modificadores de acesso foram meticulosamente escolhidos:

- **Private:** Campos como as propriedades da matriz de peças e estados de controle (`check`, `checkmate`, `turn`) são estritamente privados na classe `ChessMatch` e `Board`. Isso impede que uma classe externa altere o turno do jogo ou force um xeque-mate artificialmente.
- **Protected:** O acesso ao tabuleiro (`getBoard()`) dentro de `ChessPiece` é mantido como protegido, permitindo que somente as subclasses de peças vasculhem a matriz do tabuleiro para calcular seus movimentos possíveis, blindando esse acesso contra a camada visual (`UI`).

2.4. Polimorfismo

O polimorfismo é a engrenagem principal que confere elegância ao motor. Um exemplo claro ocorre na classe `UI`, especificamente no método `printBoard`. Este método recebe e manipula uma matriz bidimensional genérica do tipo `ChessPiece[][]`.

Em tempo de execução, através do mecanismo de *dynamic binding* (ligação dinâmica), o Java identifica se aquela referência genérica aponta para um `Pawn`, `King` ou `Knight`, invocando o respectivo método `toString()` e gerando o caractere representativo correto ("P", "K", "N") de forma transparente e modular.

Impacto do Polimorfismo: Se uma nova peça personalizada fosse adicionada ao jogo amanhã, nenhuma linha de código da classe `UI` ou dos métodos de validação genérica de movimento em `ChessMatch` precisaria ser modificada ou reescrita, evidenciando o cumprimento do princípio Open/Closed (Aberto para Extensão, Fechado para Modificação).

3. O Papel do Atributo MoveCount e Regras Temporais

A introdução da propriedade privatizada `moveCount` (junto aos métodos `increaseMoveCount()` e `decreaseMoveCount()`) na classe abstrata `ChessPiece` serve como fundação histórica para validar jogadas complexas que dependem estritamente do tempo da partida ou de estados passados.

Regra / Movimento Especial	Dependência Direta do MoveCount	Comportamento Implementado
Avanço Duplo do Peão	Exige <code>moveCount == 0</code>	O peão só pode optar por avançar duas casas à frente se for o seu primeiríssimo movimento no jogo.
Roque (Pequeno / Grande)	Exige <code>King.moveCount == 0</code> e <code>Rook.moveCount == 0</code>	A translação coordenada simultânea é sumariamente proibida se o Rei ou a Torre envolvida já tiverem se movido previamente na partida.
Mecânica de Desfazer Jogadas	Decremento controlado no <code>undoMove()</code>	Essencial para o algoritmo de teste de Xeque. O sistema simula um movimento, incrementa o contador e, caso detecte autocheque, desfaz a ação decrementando o contador para restaurar o estado original perfeitamente.

4. Implementação Vetorial e Polimórfica das Peças

A varredura de movimentos foi codificada utilizando lógica vetorial por matrizes de booleanos. A diferença crucial de comportamento reside nas naturezas de movimentação:

- **Peças de Passo/Salto (King e Knight):** Avaliam de forma pontual posições fixas ao seu redor. O Cavalo salta em "L" avaliando até 8 posições ortogonais/diagonais distintas, enquanto o Rei avalia as adjacências imediatas, sem estruturas de repetição em cascata.
- **Peças Deslizantes (Rook, Bishop e Queen):** Utilizam laços de repetição `while` que avançam vetorialmente em direções específicas até encontrar o fim do tabuleiro, uma peça aliada (bloqueio intransponível) ou uma peça adversária (permitindo a captura antes de interromper o laço). A Rainha sintetiza perfeitamente esse comportamento, unificando as 4 direções ortogonais da Torre com as 4 direções diagonais do Bispo.

5. Resolução Lógica dos Movimentos Especiais

Os movimentos especiais do xadrez quebram a estrutura padrão de "origem-destino simples", exigindo que a classe controladora `ChessMatch` atue modificando o estado periférico do tabuleiro de forma coordenada.

5.1. Roque (Castling)

O método `makeMove` monitora se a peça movida é um Rei e se o deslocamento lateral foi de duas colunas. Caso positivo, o motor calcula automaticamente a posição da respectiva Torre (Ala do Rei ou Ala da Rainha), remove-a de sua origem e a reposiciona ao lado do Rei, incrementando o contador de ambas as peças de forma atômica e indissociável.

5.2. En Passant

Uma mecânica altamente dependente do estado volátil da rodada. A classe `ChessMatch` expõe uma propriedade chamada `enPassantVulnerable`. Se um peão faz um avanço duplo, ele é marcado como a peça vulnerável daquele turno específico.

No turno subsequente imediato, os peões adversários que estão nas colunas adjacentes e na mesma linha ganham o direito polimórfico de se mover em diagonal para a casa vazia atrás do peão vulnerável. O método `makeMove` detecta esse movimento em diagonal para uma casa vazia e remove cirurgicamente o peão inimigo localizado na linha adjacente. Se a jogada não for feita naquele turno, `enPassantVulnerable` é resetado para `null`, extinguindo o direito temporal de captura.

5.3. Promoção (Promotion)

Ao concluir o movimento de um peão para a última linha da matriz (linha 0 para as Brancas ou linha 7 para as Pretas), a propriedade `promoted` da partida é populada. A camada de apresentação captura esse estado e força a escolha do novo tipo tático. O método `replacePromotedPiece` instancia de maneira polimórfica a nova peça selecionada, deleta o peão e atualiza a lista de peças ativas no tabuleiro.

6. Escolhas de Design Além das Restrições Acadêmicas

Para elevar a qualidade do projeto a um nível profissional de engenharia, foram adotadas abordagens arquiteturais autônomas que não costumam ser exigidas de forma impositiva em ambientes estritamente acadêmicos:

6.1. Separação de Camadas Orientada a Domínios (DDR)

A divisão rigorosa entre os pacotes `boardgame` e `chess` reflete o princípio da modularidade extrema. O pacote `boardgame` atua como um framework independente. Ele desconhece por completo o significado de "Xadrez", "Xeque" ou "Torre". Ele apenas gerencia uma grade matricial e colisões genéricas de peças.

Essa decisão arquitetural permite que o motor de `boardgame` seja 100% reaproveitado para implementar outros jogos de tabuleiro (como Damas ou Othello) sem alterar uma única linha de código desse pacote, demonstrando alta coesão e baixo acoplamento.

6.1. Programação Defensiva Baseada em Exceções Customizadas

Em vez de permitir que erros de entrada (como digitação de posições inexistentes ou seleções inválidas de peças adversárias) lancem exceções brutas do sistema ou resultem em falhas catastróficas em tempo de execução, foi estruturada uma malha robusta de tratamento de erros através das classes personalizadas `BoardException` e `ChessException` (ambas herdando de `RuntimeException`). Cada método atua validando pré-condições de forma defensiva antes de executar qualquer alteração de estado no sistema.

6.2. Uso Moderno de Java Streams e Expressões Lambda

A gestão das coleções de peças ativas e capturadas no tabuleiro poderia ter sido feita por meio de laços tradicionais `for`. Contudo, optou-se pela utilização do paradigma funcional moderno do Java com a *Stream API* e predicados de expressões lambda (ex: `piecesOnTheBoard.stream().filter(...)`).

Isso não apenas tornou o código mais limpo e legível, mas otimizou significativamente as rotinas críticas de cálculo de Xeque e Xeque-mate, onde o motor precisa varrer e filtrar as peças vivas do oponente repetidas vezes a cada simulação de jogada.